

II YEAR /III SEMESTER/UNIT -1

Introduction: Algorithmic notation –Programming principles –Creating programs
Analyzing programs. Arrays: One dimensional array, multidimensional array, pointer
arrays. Searching: Linear search, Binary Search, Fibonacci search. Sorting techniques:
Internal sorting - Insertion Sort, Selection Sort, Shell Sort, Bubble Sort, Quick Sort, Heap
Sort, Merge Sort and Radix Sort

Data structures, Algorithmic notation, Arrays**1. Define Abstract Data Type (Nov 2018, April 2017))**

Abstract Data type (ADT) is a type (or class) for objects whose behaviour is defined by a set of value and a set of operations.

The definition of ADT only mentions what operations are to be performed but not how these operations will be implemented

The process of providing only the essentials and hiding the details is known as abstraction.

2 .List out the performance measure of an algorithm (Nov 2018)

Algorithm Efficiency

Time efficiency - a measure of amount of time for an algorithm to execute.

Space efficiency - a measure of the amount of memory needed for an algorithm to execute.

3 .Define data structure (Nov 2019, April 2017)

- It is defined as representation of data or storing or organization of data.
- Simply it is the combination of Data structure & algorithm.
- Data structure is the structural representation of logical relationships between elements of data.
- Data structure mainly specifies the structured organization of data, by providing accessing methods with correct degree of associatively.

4. What is one dimensional array? Example (Nov 2019)

An array is a fixed size sequence collection of elements of the same data type.

It is simply a grouping of like type data.

In its simplest form an array can be used to represent a list of numbers or a list of names.

There are two types of arrays:

1. One dimensional array.
2. Multi dimensional array.

The syntax for declaring the array is:

DATATYPE ARRAYNAME [SIZE]; Where ARRAY NAME = name of the array, SIZE = size of the array

Eg: **int a[10];**

Where a- name of the array

10- size of the array.

5. What is Algorithm (Dec 2016)

- Algorithm is defined as set of instruction to perform a particular task. In Other words algorithm is a step by step procedure to perform a specific task.

6. What is time Complexity (Dec 2016, Nov 2014?)

Time complexity of an algorithm is the amount of time (or number of steps) needed by a program to complete its task (i.e. to execute a particular algorithm). The way in which the number of steps required by an algorithm varies with the size of the problem it is solving. The time taken for algorithm is comprised of two times

- COMPILATION TIME
- RUN TIME

SORTING TECHNIQUES

7. Average case and worst case of Bubble sort?

Bubble sort has a worst-case and average complexity of $O(n^2)$, where n is the number of items being sorted. Most practical sorting algorithms have substantially better worst-case or average complexity, often $O(n \log n)$ When the list is already sorted (best-case), the complexity of bubble sort is only $O(n)$.

8. What are the basic operations of data structure?

There are different types of operations that can be performed for the manipulation of data in every data structure.

1. Traversing
2. Searching
3. Sorting
4. Insertion
5. Deletion.

9. What is mean by linear and non linear data structure? Give some Example (Nov 2017)

Linear data structure is data structure having a linear relationship between its adjacent elements. Linked lists are examples of linear data structure.

Non-linear data structures are data structures that don't have a linear relationship between its adjacent elements but have a hierarchical relationship between the elements. Trees and graphs are examples of nonlinear data structures.

10. State some of the operations that can be performed on a Array (Nov 2017)**OPERATION ON ARRAYS:**

- Sorting in ascending/descending order
- Searching an element in an array
- Deleting/Inserting an element in an array
- Finding out the largest or smallest element in an array
- Removing a duplicate (repeated elements) from the array.

11. State the various algorithmic notations (Nov 2015)

1. Big-Oh Notation(O)
2. Big-Omega Notation(Ω)
3. Big-Theta Notation(θ)
4. Little-oh Notation(o)
5. Little -omega Notation(ω)

12. What is an Array?(Nov 2015)

- An array is a finite ordered and collection of homogenous data elements. All the elements of an array are of the same data type only so it was termed as collection of homogenous elements.
- An array is a kind of linear data structure because all the elements of the array are stored in linear order.

13. Arrange the following Big -oh notation terms in asymptotic ascending order. 2^n , $\log n$, n , n^2 , $n \log n$.(Sep 2020).

$\log n, n \log n, n, n^2, 2^n$

14. Convert the following from infix to postfix expression.

$A+B*CA+B*C^D^E/(E+F)*G/H+K$.(Sep 2020).

$ABCA*+BCDE^{^^}*EF+/G*H/+K+$

11 Marks

- 1. State the best case and worst case analysis of binary search (Nov 2017, nov 2015, Dec 2016, Nov 2014)**

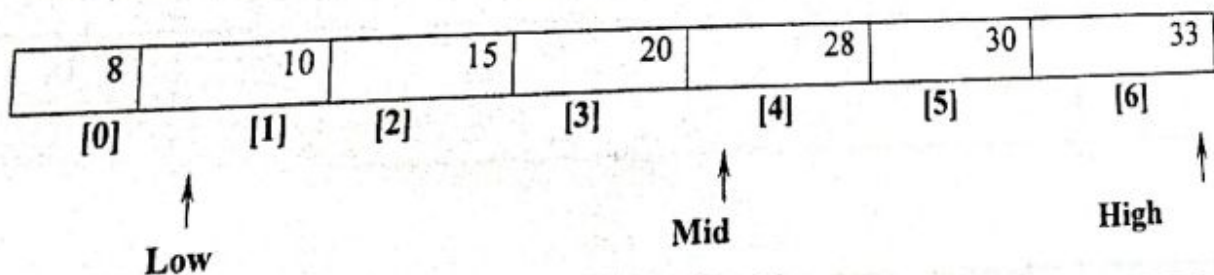
BINARY SEARCH

- The main objective of binary search is to search a particular element which is present in the list of elements. The list of elements should be in a increasing or non-decreasing or sorted order.
 - For searching lists with more values linear search is insufficient, binary search helps in searching larger lists. To search a particular item the approximate middle entry of the table is located, and its key value is examined.
1. If the search value is higher than the middle value, then the search is made with the elements after the middle element.
 2. If the search value is smaller than the middle element, then the search is made with the elements before the middle value.
 3. This process continues till the required target is found

```

Alg Binary search (a, n,x)
// a - array of integers
// n - number of elements
// x - search element
{
    low=1;
    high=n;
    while(low<=high)
    {
        mid=(low+high)/2;
        if(a[mid]==x)
            return x;
        else if(x<a[mid])
            low=1;
        high=mid-1;
        else if(x >a[mid])
            low=mid+1;
            high=n;
    }
    return 0;
}

```

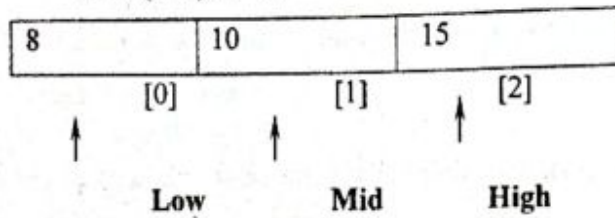


Is $15 == A[3]$? No

$15 < A[3]$, repeat the steps with $low = 0$ and $high = mid - 1 = 2$

Is $low > high$? NO

$$Mid = (0 + 2) / 2 = 1.$$

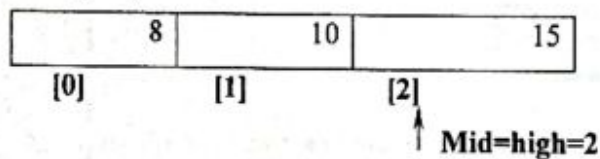


Is $15 == A[1]$? No

$15 > A[1]$, repeat the steps with $low = mid + 1 = 2$ and $high = 2$

Is $low > high$? NO

$$Mid = (2 + 2) / 2 = 2.$$



Is $15 == A[2]$? Yes

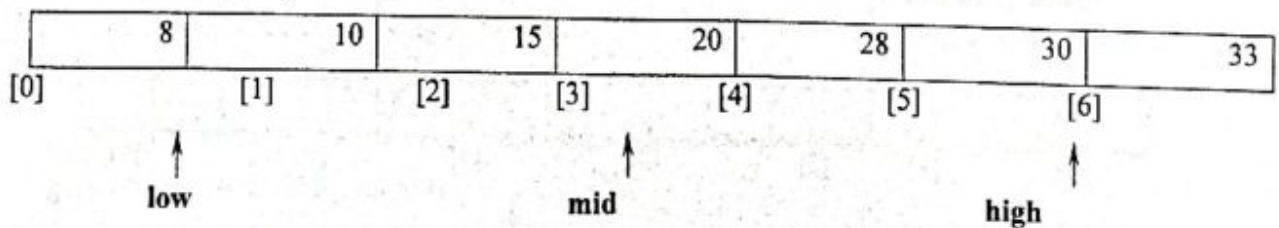
Return (2).

Element 15 is present in the position 2.

Let us search for an element that is not in the list. Search Element=9

Is $low > high$? NO

$$Mid = (0 + 6) / 2 = 3.$$

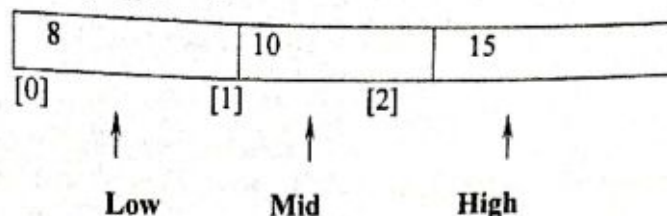


Is $9 == A[3]$? No

$9 < A[3]$, repeat the steps with $low = 0$ and $high = mid - 1 = 2$

Is $low > high$? NO

$Mid = (0 + 2) / 2 = 1$.

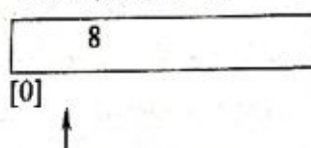


Is $9 == A[1]$? No

$9 < A[1]$, repeat the steps with $low = 0$ and $high = mid - 1 = 0$

Is $low > high$? NO

$Mid = (0 + 0) / 2 = 0$.



$Low = mid = high = 0$

Is $9 == A[0]$? No

Repeat the steps with $low = mid + 1 = 1$ and $high = 0$

Is $low > high$? Yes

When low value becomes greater than high value algorithm returns that the element 9 is not present in the list.

TIME COMPLEXITY:	BEST	WORST	AVERAGE
	$O(1)$	$O(\log n)$	$O(\log n)$

2. Explain the Quick sort algorithm (Nov 2017).

Quick Sort is a Divide and Conquer algorithm. It picks an element as pivot and partitions the given array around the picked pivot. There are many different versions of quickSort that pick pivot in different ways.

1. Always pick first element as pivot.
2. Always pick last element as pivot (implemented below)

Pick a random element as pivot.

Pick median as pivot.

The key process in quickSort is partition(). Target of partitions is, given an array and an element x of array as pivot, put x at its correct position in sorted array and put all smaller elements (smaller than x) before x, and put all greater elements (greater than x) after x. All this should be done in linear time.

Pseudo Code for recursive QuickSort function :

/* low --> Starting index, high --> Ending index */

quickSort(arr[], low, high)

```
{
    if (low < high)
    {
        /* pi is partitioning index, arr[pi] is now
           at right place */
        pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1); // Before pi
        quickSort(arr, pi + 1, high); // After pi
    }
}
```

quicksort

Partition Algorithm

There can be many ways to do partition, following pseudo code adopts the method given in CLRS book. The logic is simple, we start from the leftmost element and keep track of index of smaller (or equal to) elements as i. While traversing, if we find a smaller element, we swap current element with arr[i]. Otherwise we ignore current element.


```
/* low --> Starting index, high --> Ending index */
quickSort(arr[], low, high)
{
    if (low < high)
    {
        /* pi is partitioning index, arr[pi] is now
        at right place */
        pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1); // Before pi
        quickSort(arr, pi + 1, high); // After pi
    }
}
```

3. Explain about Multi dimensional array (Nov 2019)

- If three or more subscript or index is required to refer the elements of an array then the array is called as three dimensional or multidimensional arrays. Three dimensional arrays can be compared with books whereas two-dimensional and one-dimensional arrays can be compared with a page and a line respectively. Here three major dimensions can be termed as row, column and page.

1. Number of rows= X(number of elements in a column)

2. Number of columns=Y(number of elements in a row)

3 .Number of pages=Z

DECLARATION OF AN ARRAY:

The declaration can be as follows:

Example Program based on 3-D Arrays:

```
void main()
{
    int mark[2][4][3],i,j,k;
    int totmark,semsum,papsum;

    clrscr();

    for(i=0;i<2;i++)
    {
        printf("For the %d semester \n",i+1);

        for(j=0;j<4;j++)
```

```

        {
            printf("\n Enter 3 marks for paper %d\n",j+1);
            for(k=0;k<3;k++)
                scanf("%d",&mark[i][j][k]);
        }
    }
    totmark=0;
    for(i=0;i<2;i++)
    {
        semsum=0;
        printf("\n Semester %d\n",i+1);
        printf("Paper marks \n");
        for(j=0;j<4;j++)
        {
            papsum=0;
            for(k=0;k<3;k++)
                papsum+=mark[i][j][k];
            printf("%d   %d\n",j+1,papsum);
            semsum+=papsum;}
        printf("Total marks in %d semester = %d \n",i+1,semsum);
        totmark+=semsum;}
    printf("\n Grand total = %d",totmark);
    getch();

```

4. Explain in detail about Merge Sort (Nov 2017)

Merge Sort is a Divide and Conquer algorithm. It divides the input array into two halves, calls itself for the two halves, and then merges the two sorted halves. The merge() function is used for merging two halves. The merge(arr, l, m, r) is a key process that assumes that arr[l..m] and arr[m+1..r] are sorted and merges the two sorted sub-arrays into one. See the following C implementation for details.

MergeSort(arr[], l, r)

If $r > 1$

1. Find the middle point to divide the array into two halves:

$$\text{middle } m = l + (r-l)/2$$

2. Call mergeSort for first half:

Call mergeSort(arr, l, m)

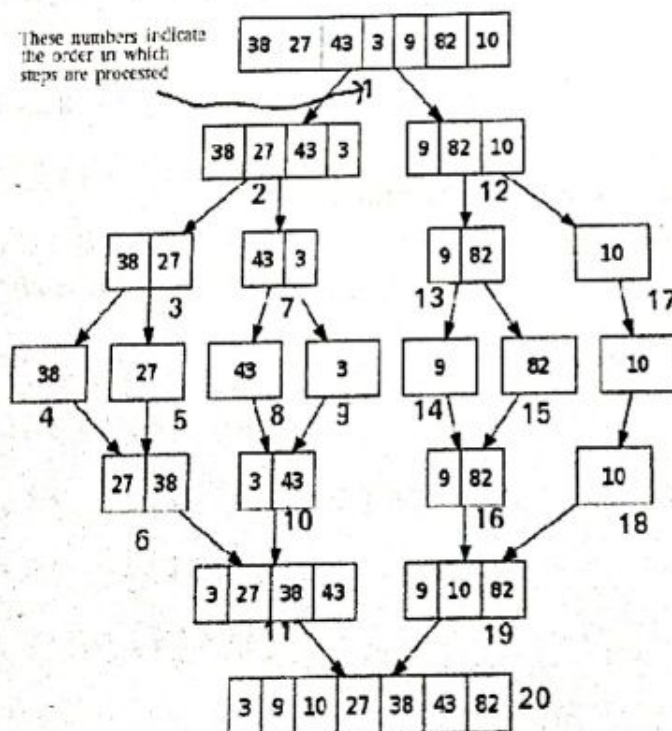
3. Call mergeSort for second half:

Call mergeSort(arr, m+1, r)

4. Merge the two halves sorted in step 2 and 3:

Call merge (arr, l, m, r)

The following diagram shows the complete merge sort process for an example array {38, 27, 43, 3, 9, 82, 10}. If we take a closer look at the diagram, we can see that the array is recursively divided in two halves till the size becomes 1. Once the size becomes 1, the merge processes come into action and start merging arrays back till the complete array is merged.



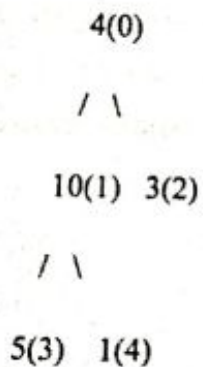
5. What is heap sort? Explain with an Example (Dec 2016)

Heap sort is a comparison based sorting technique based on Binary Heap data structure. It is similar to selection sort where we first find the maximum element and place the maximum element at the end. We repeat the same process for the remaining elements.

Heap Sort Algorithm for sorting in increasing order:

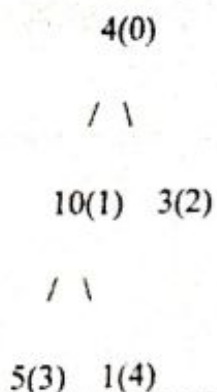
1. Build a max heap from the input data.
2. At this point, the largest item is stored at the root of the heap. Replace it with the last item of the heap followed by reducing the size of heap by 1. Finally, heapify the root of the tree.
3. Repeat step 2 while size of heap is greater than

Input data: 4, 10, 3, 5, 1

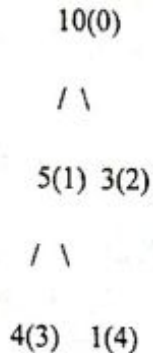


The numbers in bracket represent the indices in the array representation of data.

Applying heapify procedure to index 1:



Applying heapify procedure to index 0:



The heapify procedure calls itself recursively to build heap in top down manner.

6. Explain Bubble sort and Quick sort with example (April 2017)

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order.

Example:

First Pass:

(5 1 4 2 8) $\rightarrow$ (1 5 4 2 8), Here, algorithm compares the first two elements, and swaps since $5 > 1$.

(1 5 4 2 8) $\rightarrow$ (1 4 5 2 8), Swap since $5 > 4$

(1 4 5 2 8) $\rightarrow$ (1 4 2 5 8), Swap since $5 > 2$

(1 4 2 5 8) $\rightarrow$ (1 4 2 5 8), Now, since these elements are already in order ($8 > 5$), algorithm does not swap them.

Second Pass:

(1 4 2 5 8) $\rightarrow$ (1 4 2 5 8)

(1 4 2 5 8) $\rightarrow$ (1 2 4 5 8), Swap since $4 > 2$

(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)

(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)

Now, the array is already sorted, but our algorithm does not know if it is completed. The algorithm needs one whole pass without any swap to know it is sorted.

Third Pass:

(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)

(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)

(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)

(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)

QuickSort is a Divide and Conquer algorithm. It picks an element as pivot and partitions the given array around the picked pivot. There are many different versions of quickSort that pick pivot in different ways.

1. Always pick first element as pivot.
2. Always pick last element as pivot (implemented below)
3. Pick a random element as pivot.
4. Pick median as pivot.

The key process in quickSort is partition(). Target of partitions is, given an array and an element x of array as pivot, put x at its correct position in sorted array and put all smaller elements (smaller than x) before x , and put all greater elements (greater than x) after x . All this should be done in linear time.

7. Explain the Asymptotic notation (April 2017)

ASYMPTOTIC NOTATIONS

Complexity of an algorithm is commonly represented by following asymptotic notations,

1. $\Theta(x)$: Means that the rate varies directly with that of the constant x .
2. $\Theta(n)$: Means that the rate varies directly with size of the input.
3. $\Theta(n^2)$: Means that the rate varies with the square of the size of the input.
4. $\Theta(n^3)$: Means that the rate varies with the cube of the size of the input.
5. $\Theta(n!)$: Means that the rate varies with the factorial of the size of the input.
6. $\Theta(2^n)$: Means that the rate varies exponentially with the size of the input.
7. $\Theta(\log n)$: Means that the rate varies with the logarithm (power to which 2 has to be raised to get n) of the size of the input.
8. $\Theta(n \log n)$: Means that the rate varies n times with the logarithm of the size of the input.

The following notations are the methods used to estimate the efficiency of an algorithm:

1. Big-Oh Notation (O)
2. Big-Omega Notation (Ω)

3. Big-Theta Notation(θ)
4. Little-oh Notation(o)
5. Little -omega Notation(ω)

8. State the Insertion sort with an Example. (Nov 2015)

Insertion sort is a simple sorting algorithm that works similar to the way you sort playing cards in your hands. The array is virtually split into a sorted and an unsorted part. Values from the unsorted part are picked and placed at the correct position in the sorted part.

Algorithm

To sort an array of size n in ascending order:

- 1: Iterate from $\text{arr}[1]$ to $\text{arr}[n]$ over the array.
- 2: Compare the current element (key) to its predecessor.
- 3: If the key element is smaller than its predecessor, compare it to the elements before. Move the greater elements one position up to make space for the swapped element.

Example:

Insertion-sort

Another Example:

12, 11, 13, 5, 6

Let us loop for $i = 1$ (second element of the array) to 4 (last element of the array)

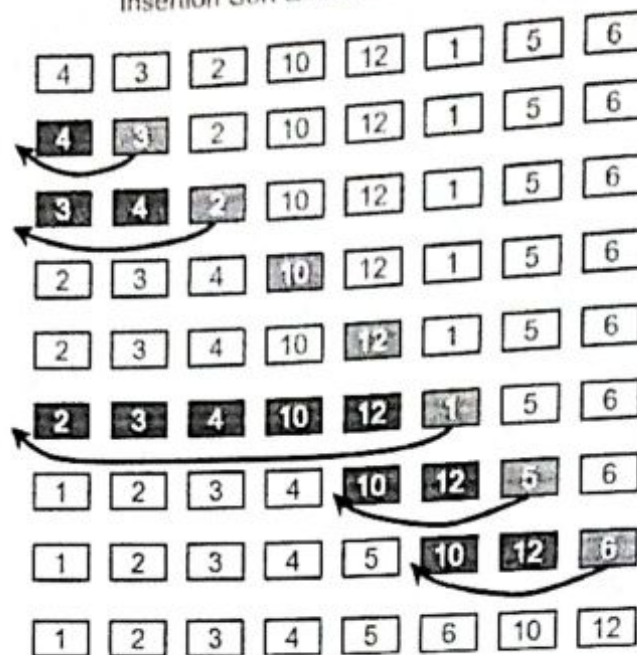
$i = 1$. Since 11 is smaller than 12, move 12 and insert 11 before 12
11, 12, 13, 5, 6

$i = 2$. 13 will remain at its position as all elements in $A[0..i-1]$ are smaller than 13
11, 12, 13, 5, 6

$i = 3$. 5 will move to the beginning and all other elements from 11 to 13 will move one position ahead of their current position.
5, 11, 12, 13, 6

$i = 4$. 6 will move to position after 5, and elements from 11 to 13 will move one position ahead of their current position.
5, 6, 11, 12, 13

Insertion Sort Execution Example



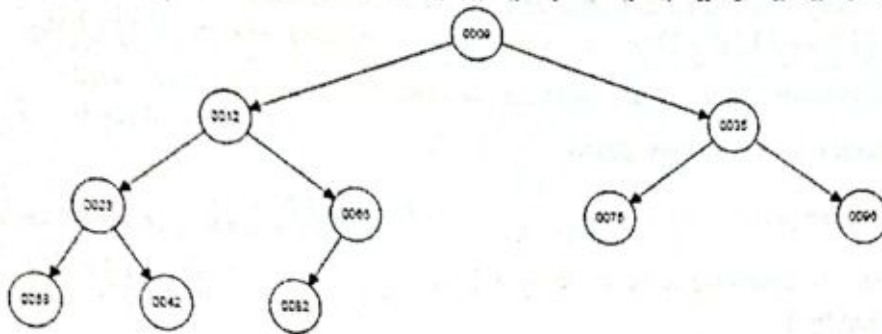
9. What is Heap sort? Give an algorithm for Heap sort. What is its time complexity? 42,23,75,12,66,35,96,58,9,82.(Sep 2020)

Heap sort is a comparison based sorting technique based on Binary Heap data structure. It is similar to selection sort where we first find the maximum element and place the maximum element at the end. We repeat the same process for the remaining elements.

Let us first define a Complete Binary Tree. A complete binary tree is a binary tree in which every level, except possibly the last, is completely filled, and all nodes are as far left as possible (Source Wikipedia)

A Binary Heap is a Complete Binary Tree where items are stored in a special order such that value in a parent node is greater(or smaller) than the values in its two children nodes. The former is called as max heap and the latter is called min-heap. The heap can be represented by a binary tree or array.

Heap Sort has $O(n \log n)$ time complexities for all the cases (best case, average case, and worst case). ... Also it performs sorting in $O(1)$ space complexity.



UNIT-II

Stacks: Definition – operations – applications of stack. Queues: Definition – operations – Priority queues – De queues – Applications of queue. Linked List: Singly Linked List, Doubly Linked List, Circular Linked List, linked stacks, Linked queues, Applications of Linked List – Dynamic storage management – Generalized list.

1. List the stack operation(Nov 2018)

In Stack, we can perform two operations namely Push and Pop.

Push: Push means inserting a new element into the stack. Insertion can be done by incrementing top by 1.

Pop: Pop means deleting an element from the stack. Deletion can be done by decrementing top by 1.

2.State the Advantage of linked list over array(Nov 2018)

Arrays	Linked List
➤ Size of any arrays is fixed.	➤ Size of a list is variable.
➤ It is necessary to specify the number of elements during the declaration.	➤ It is not necessary to specify the number of elements during the declaration.
➤ It occupies less memory space than linked list for the same memory number of elements.	➤ It occupies more memory space.

3. Difference between queue and linked list(Nov 2018)

Queue	Linked List
➤ A Queue is an ordered collection of items, where all insertions are made at the end of the sequence called Rear and all deletions always are made from the beginning of the sequence called Front.	➤ Size of a list is variable.
➤ In Queue, we get data items out in same order compared to the order they have been put into the queue.	➤ It is not necessary to specify the number of elements during the declaration.
➤ Queue is otherwise called as FIFO.	➤ It occupies more memory space.

4. What is a list? (Dec 2016)

- A *generalized list*, A , is a finite sequence of $n > 0$ elements, $l_1, \dots, l_n$ where the l_i are either atoms or lists.
- The elements l_i in A which are not atoms are said to be the *sub lists* of A .
- The list A itself is written as $A = (l_1, \dots, l_n)$. A is the *name* of the list $(l_1, \dots, l_n)$ and n its *length*.
- By convention, all list names will be represented by capital letters. Lower case letters will be used to represent atoms. If $n = 1$, then l_1 is the *head* of A while $(l_2, \dots, l_n)$ is the *tail* of A .

5. State the Application of stack (Dec 2016, Nov 2015)

Some of the applications of Stack are:

- i. Matching of nested parenthesis in a mathematical expression.
- ii. Conversion of infix to postfix.
- iii. Evaluation of postfix.

6. Write the advantage of linked list

The advantages in using a linked list are

- It is not necessary to specify the number of elements in a linked list during its declaration
- Linked list can grow and shrink in size depending upon the insertion and deletion that occurs in the list.
- Insertions and deletions at any place in a list can be handled easily and efficiently
- A linked list does not waste any memory space.

7. Convert into reverse polish notation $(a-b)(c(d+e-f(g/h)))$

$a\ b\ -\ c\ d\ e\ +\ f\ g\ h\ /\ -$

8. Write the different way to implement the List (April 2017)

There are two general-purpose List implementations — Array List and Linked List. Most of the time, you'll probably use Array List, which offers constant-time positional access and is just plain fast. It does not have to allocate a node object for each element in the List, and it can take advantage of System.

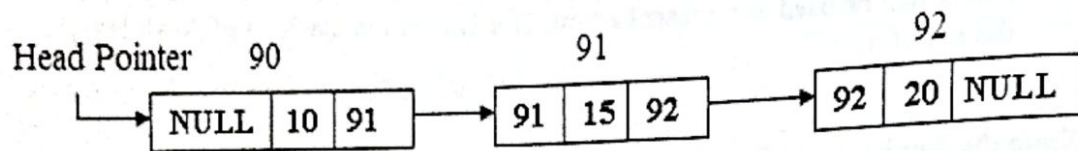
9. Evaluate the value of expression in postfix notation $752+*415-/-$ (Nov 2017)

The value of the above postfix conversion is -48.

10. Explain the various structures of queue (Nov 2017)

1. Enqueue
2. Dequeue

11. Draw the node structure of doubly linked list (Nov 2015)



12. What is de-queue? What are its variants?(Sep 2020)

- De queue is the short form of double ended queue
 - In which we can insert or delete the elements either at front or rear
- Its variant are

1. Input restricted Dequeue:
2. Output restricted dequeues

13. Define Linear List (Sep 2020)

The elements are ordered within the linear list in a linear sequence. ... Linear lists are usually simply denoted as lists. Unlike an array, a list is a data structure allowing insertion and deletion of elements at an arbitrary position of the sequence.

11 Marks

Linked List: Singly Linked List, Doubly Linked List, Circular Linked List

1. Explain Array and linked list representation of queue.

we will implement the queue using a linked list as well as with an array.

We will maintain two pointers - tail and head to represent a queue. head will always point to the oldest element which was added and tail will point where the new element is going to be added.

head and tail in queue using array

To insert any element, we add that element at tail and increase the tail by one to point to the next element of the array.

Inserting element in queue using array

Suppose tail is at the last element of the queue and there are empty blocks before head as shown in the picture given below.

Circular order in queue using array

In this case, our tail will point to the first element of the array and will follow a circular order.

Tail at 1 in circular order for queue using array

Initially, the queue will be empty i.e., both head and tail will point to the same location i.e., at index 1. We can easily check if a queue is empty or not by checking if head and tail are pointing to the same location or not at any time.

Empty queue using an array

Stacks: Definition – operations - applications of stack.

2. Explain Stack application Evaluation of expression with example(Nov 2018)(Nov 2017).

STACK APPLICATIONS

- Infix into postfix Expression.
- Expression Evaluation.

EVALUATING POSTFIX EXPRESSION:

Once the expression has been parsed into postfix form, another stack can be used to evaluate postfix expression. As an example, consider the postfix expression,

AB*CDE/-+ \0

Let us suppose that A, B, C, D and E are the symbols used associated with values,

A=5, B= 3, C=6, D=8 & E=2.

To evaluate such an expression, we repeatedly read characters from the postfix expression. If the character read is an operand, push the value associated with it onto the stack. If it is an operator, pop two values from the stack, apply the operator to them and push the result back on the stack. The process is illustrated in the following figure.

1. Read A 2. Read B 3. Read * (Pop 2 elements Perform opn and push res in stack)
4. Read C

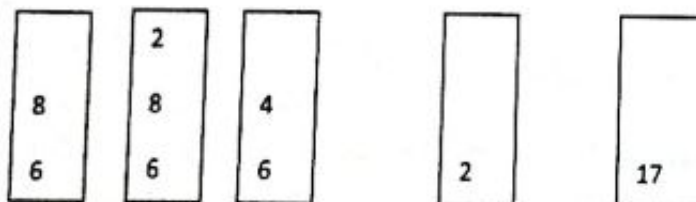


5. Read D

6. Read E

7. Read / (Pop 2 elements Perform opn and push res in stack)

8. Read - 9. Read + 10. Read \0



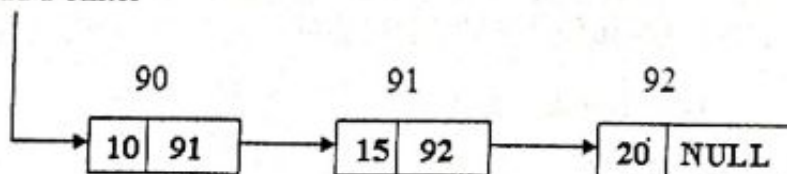
End of postfix expression.

Answer is on top of stack

3. Explain in detail about linked list. (Nov 2019, Nov 2014)

- In the single linked list each link has single link to the next node.
- It is otherwise called as linear linked list.
- It contains the head pointer which holds the address of the first node.
- Using head pointer only we can access entire linked list.
- The below diagram shows the single linked

Head Pointer



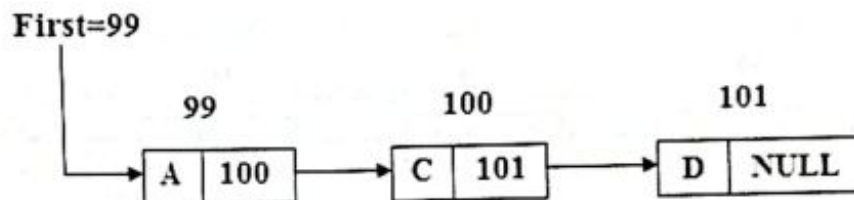
- In single linked list we can traverse in one direction from head to null.
- We cant move in reverse direction i.e. from null to head.
- Link field of last node have the null pointer indicates the end of the list.

Operations In Single Linked List:

- We can insert and delete the node in a single linked list.

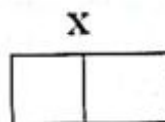
Inserting The Node Into A Single Linked List:

Consider the linked list:



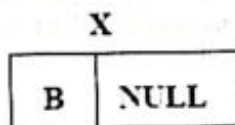
Where first is a pointer which holds the address of the first node.

If we want to insert a data B , into the list , first we need the empty node with empty data and link field.



Consider x is the address of new node.

Now put the data B into the data field of a new node and put NULL into the link field.

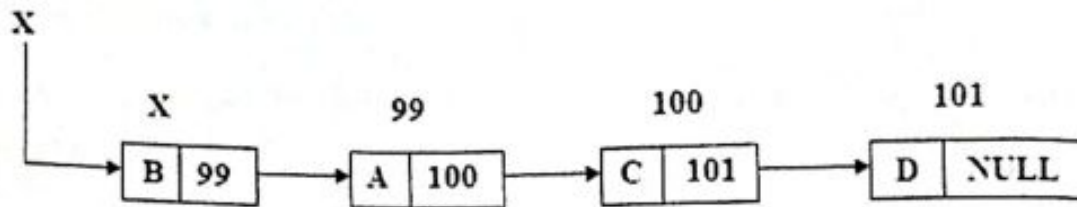


We can insert the new node by any one of the following position:

1. As a first node
2. As a intermediate node
3. As a last node.

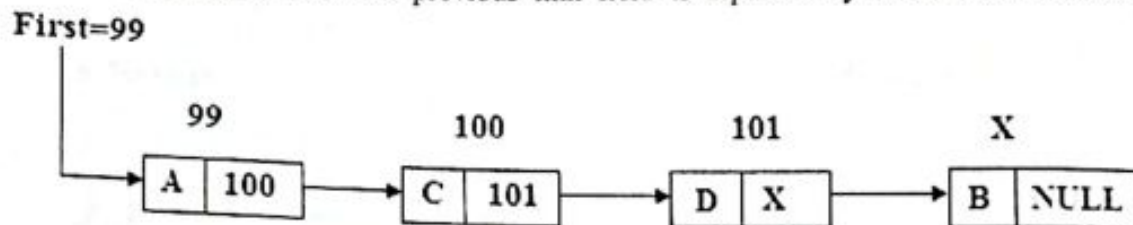
Inserting as a first node:

If we want to insert a new node as a first node, the link field of new node is replaced by FIRST and FIRST is replaced by X as shown below.



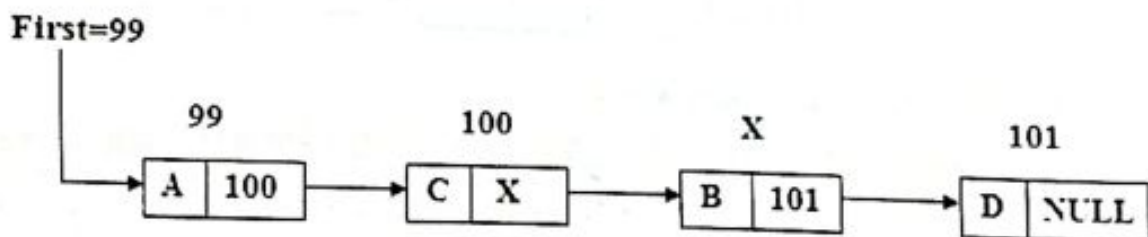
Inserting as a last node:

If we want to insert a node as a last node, the link field of new node is replaced by NULL and the link field of last but previous link field is replaced by X as shown below.



Inserting as an intermediate node:

- If we want to insert a new node as the intermediate node, first we need the address of the previous node.
- Now insert the new node in between the respective node as shown below
- If we want to insert a new node X as intermediate node, insert in between C and D.
- The link field of 99 is 100, the link field of 100 is 101 and the link field of 101 is 102.



4. Why we need Dynamic memory management (Nov 2014)

When we do not know how much amount of memory would be needed for the program beforehand.

When we want data structures without any upper limit of memory space.

When you want to use your memory space more efficiently. Example: If you have allocated memory space for a 1D array as array[20] and you end up using only 10 memory spaces then the remaining 10 memory spaces would be wasted and this wasted memory cannot even be utilized by other program variables.

Dynamically created lists insertions and deletions can be done very easily just by the manipulation of addresses whereas in case of statically allocated memory insertions and deletions lead to more movements and wastage of memory.

When you want you to use the concept of structures and linked list in programming, dynamic memory allocation is a must.

5. Explain about the push and pop operation in stack (Dec 2016)

STACK

- A stack is an ordered collection of data items.
- New data items may be inserted or deleted in a stack.
- But these insertion (PUSH) and deletion (POP) operation can be done only at one end called the top of the stack.
- Top is a pointer which always point to the top of the stack.
- The last element inserted in a stack is the first element to be removed.
- Stack is also referred to as Last in First out (LIFO) lists.

PUSH OPERATION:

- Push operation inserts an element onto the stack.
- An attempt to push an element onto the stack , when the stack is full, causes an overflow.

Push operation involves:

1. Check whether the stack is full before attempting to push an element to the stack.
2. Increment the top pointer.
3. Push the element onto the top of the stack.

ALGORITHM:

STACK - Array to hold elements

N – Total number of elements

TOP – Denotes the top element in the stack

Item – The element to be inserted at the top of a stack

1. if(TOP>=N) [Check for stack overflow]

Then CALL STACK_FULL

Exit

2. TOP<-TOP+1 [Increment TOP]

3. STACK [TOP]<-Item [Insert element]

End PUSH

POP OPERATION:

- POP operation removes an element from the stack.
- An attempt to pop an element from the stack, when the array is empty, causes an underflow.

Pop operation involves:

1. Check whether the stack is empty before attempting to pop an element from the stack.
2. Decrement the top pointer.
3. Pop the element from the top of the stack.

ALGORITHM:

STACK – Array to hold elements

TOP – Denotes the top element in the stack

Item – The element to be deleted from the top of the stack

1. If (TOP<=0) [Check for underflow on stack]

Then Call STACK_EMPTY - Exit

2. Item<-STACK [TOP] [Return top element of the stack]

3. TOP<-TOP-1 [Decrement top pointer]

If stack is implemented using array it cannot grow dynamically where as Linked stack can grow dynamically.

6. What is doubly linked list (Dec 2016, April 2017, Nov 2017)

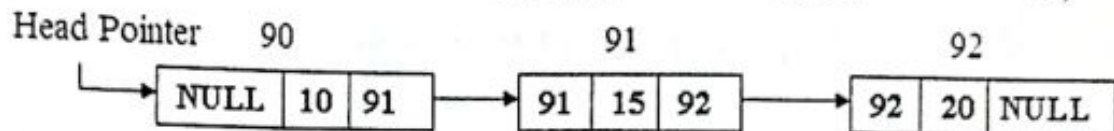
- Each node consist of three fields namely

i. Previous address field or Backward link field,

- ii. Data field or Information field,
- iii. External address field or Forward link field.

- The previous address field holds the address of the previous node and the next address field holds the address of next node.
- In Double Linked list, we can move in both the direction from head pointer to Null address or vice versa.

- Consider the following linked list,



Operations of Double Linked list:

We can perform the two operations in double linked list:

- i. Inserting
- ii. Deleting

Inserting a node into the double linked list:

- If we want to insert a node into the double linked list, we need an empty node.
- Before inserting we should check whether the stack is underflow or not.
- If AVAIL=NULL, we can't get the empty node from the stack and we have to exit from the program.
- If AVAIL!= NULL, we can get the empty node from the stack .
- **Algorithm for inserting a node into the double linked list:**

DOUINS (L, R, X, M)

1. [CHECK UNDERFLOW]
IF AVAIL= NULL THEN
WRITE ("STACK UNDERFLOW")
RETURN EXIT
2. [OBTAIN THE ADDRESS OF THE NEW NODE]
NEW<- AVAIL
3. [REMOVE THE NEW NODE FROM THE STACK]
AVAIL<- LINK (AVAIL)
4. [FILL INFO FIELD OF NEW]
IF R=NULL THEN

LPTR (NEW) <- RPTR (NEW) <-NULL

L<-R<-NEW

5. [INSERT INTO EMPTY LIST]

IF R= NULL THEN

LPTR (NEW) <-RPTR (NEW)<-NULL

L<-R<-NEW

RETURN

6. [INSERTAS LEFT MOST NODE]

LPTR (NEW) <- NULL

RPTR (NEW) <-M

LPTR (M) <-NEW

L<-NEW

7. [INSERT IN MIDDLE]

LPTR (NEW) <-LPTR (M)

RPTR (NEW) <-M

LPTR (M) <-NEW

RPTR (LPTR (NEW)) <-NEW

7. How stack is useful to evaluate the Expression(Nov 2014, April 2017)

EXPRESSION

Usual algebraic notation is often termed infix notation. Here, the arithmetic operator appears between the two operands to which it is being applied. There are two alternate notations for expressing the expression. These are prefix and postfix expressions.

Example:

A + B INFIX

+ A B PREFIX

A B + POSTFIX

The prefixes "pre -", "post -" and "in -" refer to the relative position of the operator with respect to the two operands. In prefix notation, the operator precedes the two operands, in postfix notation, the operator follows the two operands and in infix notation, the operator

is between the operands.

To understand the expression, we must first decide the order of evaluation of an expression. In C language, the operator with highest priority is evaluated first. Each operator is given certain priority.

Priority of arithmetic operators in C:

* / High

+ - Low

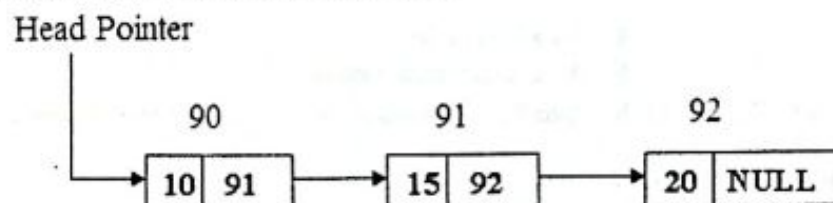
If the expression consists of more than one operator at the same level, then it follows left associativity rule. I.e., operator at left most will be done first and so evaluated from left to right.

To implement the conversion we make use of stack because it is easier to implement the concept.

8. Explain about single linked list in detail (Nov 2017)

Develop the algorithm for inserting a node as first node.

- In the single linked list each link has single link to the next node.
- It is otherwise called as linear linked list.
- It contains the head pointer which holds the address of the first node.
- Using head pointer only we can access entire linked list.
- The below diagram shows the single linked



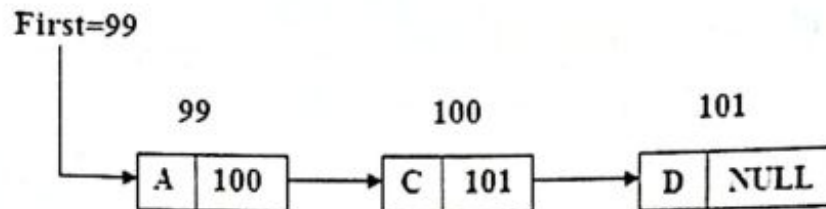
- In single linked list we can traverse in one direction from head to null.
- We cant move in reverse direction i.e. from null to head.
- Link field of last node have the null pointer indicates the end of the list.

Operations In Single Linked List:

- We can insert and delete the node in a single linked list.

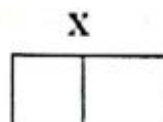
Inserting the Node into a Single Linked List:

Consider the linked list:



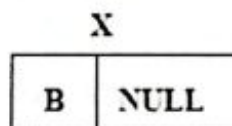
Where first is a pointer which holds the address of the first node.

If we want to insert a data B , into the list , first we need the empty node with empty data and link field.



Consider x is the address of new node.

Now put the data B into the data field of a new node and put NULL into the link field.

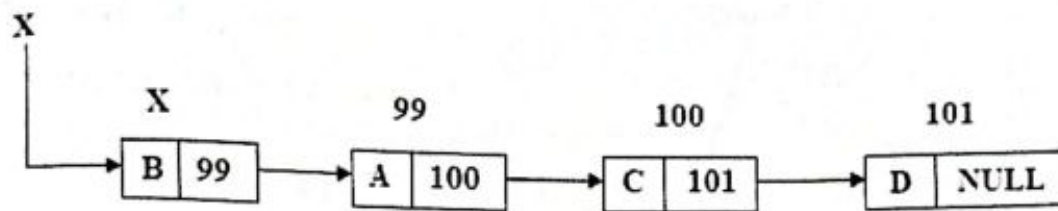


We can insert the new node by any one of the following position:

4. As a first node
5. As a intermediate node
6. As a last node.

Inserting as a first node:

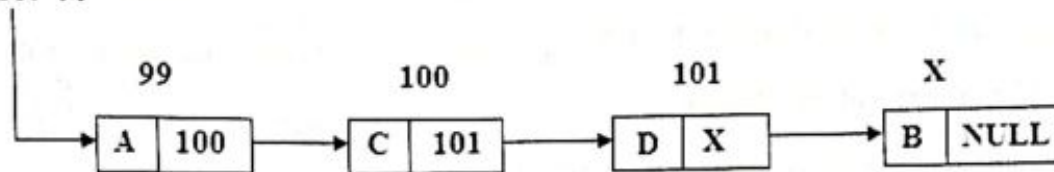
If we want to insert a new node as a first node, the link field of new node is replaced by FIRST and FIRST is replaced by X as shown below.



Inserting as a last node:

If we want to insert a node as a last node, the link field of new node is replaced by NULL and the link field of last but previous link field is replaced by X as shown below.

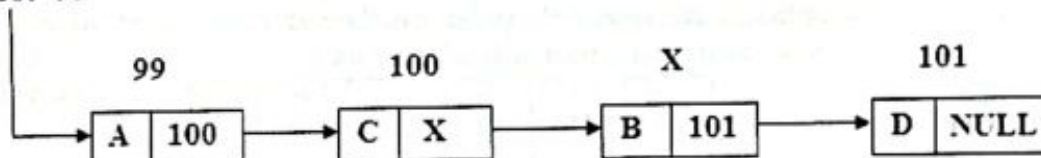
First=99



Inserting as an intermediate node:

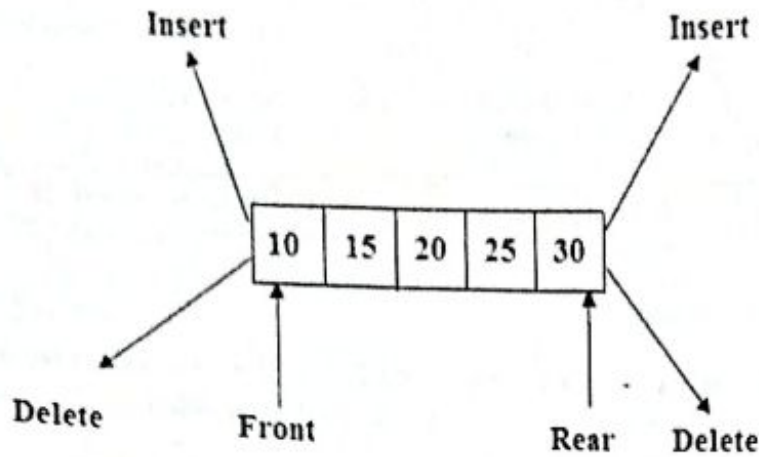
- If we want to insert a new node as the intermediate node, first we need the address of the previous node.
- Now insert the new node in between the respective node as shown below
- If we want to insert a new node X as intermediate node, insert in between C and D.
- The link field of 99 is 100, the link field of 100 is 101 and the link field of 101 is 102.

First=99



9. State the operations performed on queue (ENQUEUE, DEQUEUE)(Nov 2017,Nov 2015)

- Dequeue is the short form of double ended queue
- In which we can insert or delete the elements either at front or rear
- The below diagram shows the double ended queue.



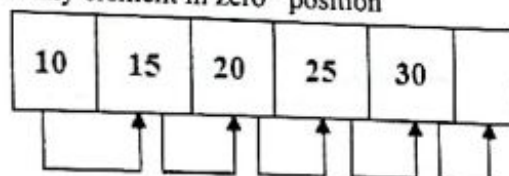
Insert new element into Dequeue:

At the end of the Dequeue:

- First check the Dequeue is full or not.
- We have to check whether the element is first to be added or inserted if it is true assign front and rear is 0.
- Now insert a element.

At the beginning of the Dequeue:

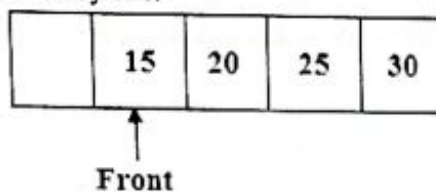
- First check the Dequeue is full or not.
- We have to check whether the element is first to be inserted if it is true then perform the following steps
 1. Shift the elements from left to right
 2. So that we need the number of elements present in the Dequeue, the position of the last element i.e. the position of the rear
 3. Now insert any element in zeroth position



Deleting the element from Dequeue:

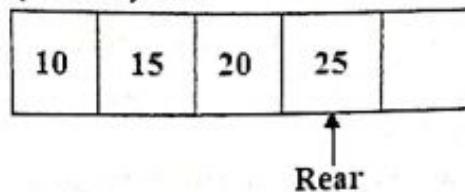
At the beginning of Dequeue:

- Delete the first element from the front position of the Dequeue
- The index of next element is stored in front pointer.
- Increment the front pointer by one.

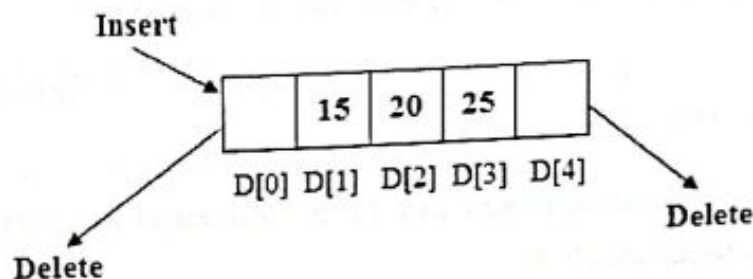


At the end of Dequeue:

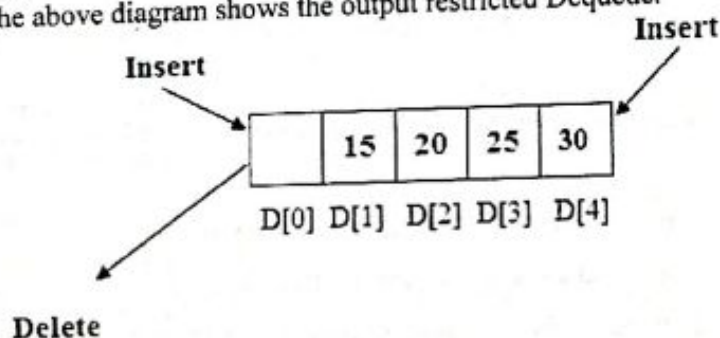
- The last element is deleted.
- The index of next element is stored in rear pointer
- Decrement the rear pointer by one.

**Input restricted Dequeue:**

- It means we can insert the element only at one end and delete the elements at both ends.
- The above diagram shows the input restricted Dequeue.

**Output restricted dequeues:**

- It means we can insert the elements in both ends and delete the elements at only one end.
- The above diagram shows the output restricted Dequeue.

**10. Explain about circular linked list (Nov 2015).**

Circular linked list is a linked list where all nodes are connected to form a circle. There is no NULL at the end. A circular linked list can be a singly circular linked list or doubly circular linked list.

Advantages of Circular Linked Lists:

- 1) Any node can be a starting point. We can traverse the whole list by starting from any point. We just need to stop when the first visited node is visited again.
- 2) Useful for implementation of queue. Unlike this implementation, we don't need to maintain two pointers for front and rear if we use circular linked list. We can maintain a pointer to the last inserted node and front can always be obtained as next of last.
- 3) Circular lists are useful in applications to repeatedly go around the list. For example, when multiple applications are running on a PC, it is common for the operating system to put the running applications on a list and then to cycle through them, giving each of them a slice of time to execute, and then making them wait while the CPU is given to another application. It is convenient for the operating system to use a circular list so that when it reaches the end of the list it can cycle around to the front of the list.
- 4) Circular Doubly Linked Lists are used for implementation of advanced data structures like Fibonacci Heap.

11. Give an algorithm to add and delete elements of a circular queue in array representation.(Sep 2020):

- **Circular queue** is another form of a linear queue in which the last position is connected to the first position of the list.
- The circular queue is similar to linear queue has two ends, the front end and the rear end.
- The rear end is where we insert the elements and the front end is where we delete the elements.
- The traversing is in only one direction (i.e., from front to rear).
- Initially the front and rear ends are at same position(i.e., -1);
- While inserting elements the rear pointer moves one by one (where front pointer doesn't change) until the front end is reached.
- If the next position of the rear is front, the queue is said to be fully occupied.
- Beyond this insertion is not done.
- But if we delete any data, we can insert the element accordingly.
- While deleting the elements the front pointer moves one by one (where as the rear point doesn't change) until the rear point is reached.

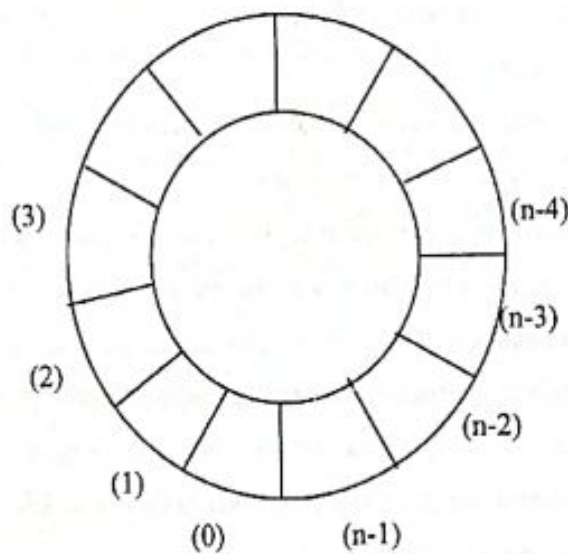
- If the front pointer reaches the rear pointer, both the *ir* positions are initialized to -1, and the queue is said to be empty.
- A more efficient queue representation is obtained by the circular array.
- It becomes more convenient to declare the array as $Q[n-1]$.
- When $rear = n-1$, the next element is entered at $Q[0]$ in case that position is free.
- Using the same conventions, front will always point one position counterclockwise from the first element in the queue.
- Again, $front=rear$ if and only if the queue is empty. Initially we have $front = rear = 1$.
- The following figure illustrates some of the possible configurations for a circular queue containing the four elements with $n>4$.

❖ $front = 0 ; rear=4$

- The assumption of circularity changes the ADD and DELETE algorithm slightly.
- In order to add an element , it will be necessary to move *rear* one position clockwise, i.e.,

if $rear = n-1$ **then** $rear = 0$
else $rear = rear+1$.

- Using modulo operator which computes remainders, this is just $rear=(rear+1)\bmod n$.
- Similarly, it will be necessary to move **front** one position clockwise each time a deletion is made.
- Again, using the modulo operation, this can be accomplished by $front=(front+1)\bmod n$.
- An examination of the algorithms indicates that addition and deletion can now be carried out in a fixed amount of time or $O(1)$.



❖ $front = n-4, rear = 0$

Procedure *ADDQ* (*item*, *Q*, *n*, *front*, *rear*)

//insert item in the circular queue stored in *Q* (0: *n*-1);

rear points to the last item and *front* is one position counter clockwise from the first item in *Q*//

$rear = (rear+1) \bmod n$ //advance rear clockwise//

if *front* = *rear* **then call** QUEUE-FULL

Q(*rear*)= *item* //insert new item //

end *ADDQ*.

Procedure *DELETEQ* (*item*, *Q*, *n*, *front*, *rear*)

//removes the front element of the queue *Q* (0: *n*-1)//

if *front* = *rear* **then call** QUEUE-EMPTY

$front = (front + 1) \bmod n$ //advance front clockwise//

item = *Q*(*front*) //set item to front of queue//

end *DELETEQ*